

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
Módulo 7. HTTP y Consumo de APIs (Smocker)	1
HTTP y Consumo de APIs	1
Uso de librerías httpx y HTTP/2	3
Uso de timeouts, reintentos y errores	4
Streaming de Respuestas	5
Smocker	6
Preparación del Laboratorio	7

NIVEL FUNDAMENTAL

MÓDULO 7. HTTP Y CONSUMO DE APIS (SMOCKER)

HTTP Y CONSUMO DE APIS

HTTP. *Protocolo* para intercambiar información entre clientes y servidores.

Cliente → *Solicitud HTTP* → Servidor
 Cliente ← *Respuesta HTTP* ← Servidor

Solicitud incluye: **Respuesta incluye:**

- | | |
|---|---|
| <ul style="list-style-type: none"> ▪ <i>Método.</i> ▪ <i>URL.</i> ▪ <i>Encabezados.</i> ▪ <i>Parámetros.</i> ▪ <i>Cuerpo (opcional).</i> | <ul style="list-style-type: none"> ▪ <i>Código de estado.</i> ▪ <i>Encabezados.</i> ▪ <i>Cuerpo.</i> ▪ <i>Información sobre el resultado.</i> |
|---|---|

Métodos HTTP principales y *códigos* de estado:

Métodos HTTP		Códigos de estado	
GET	<i>Obtener información.</i>	2xx	<i>Operación exitosa.</i>
POST	<i>Crear o enviar información.</i>	3xx	<i>Redirección.</i>
PUT	<i>Reemplazar un recurso.</i>	4xx	<i>Error relacionado con la solicitud.</i>
PATCH	<i>Modificar parcialmente.</i>	5xx	<i>Error del servidor.</i>
DELETE	<i>Eliminar un recurso.</i>		

Librería `requests`. Librería popular para realizar solicitud HTTP *síncronas*.

```
import requests
respuesta = requests.get("https://example.com")
print(respuesta.status_code) # Código HTTP devuelto por server
print(respuesta.text)
respuesta.raise_for_status() # Control de errores
```

Los errores se deben controlar. Si obtiene por respuesta un código `4xx` o `5xx`, se lanza una *excepción*.

Librería `httpx`. Ofrece una API moderna y soporta: operaciones *síncronas* y *asíncronas*, HTTP/2, timeouts configurables y streaming.

Uso síncrono:

```
import httpx

respuesta =
httpx.get("https://example.com")
print(respuesta.status_code)
```

Uso recomendado con cliente:

```
with httpx.Client() as cliente:
    respuesta =
cliente.get("https://example.com")
respuesta.raise_for_status()
```

Donde `httpx.Client`, siendo función de la biblioteca `httpx`, crea una instancia de cliente HTTP para conservar *configuración compartida*.

Librería `aiohttp`. Principalmente para clientes y servidores *asíncronos*.

```
import aiohttp
import asyncio

async def consultar():
    async with aiohttp.ClientSession() as sesion:
        async with sesion.get("https://example.com") as respuesta:
            texto = await respuesta.text()
            print(texto)
asyncio.run(consultar())
```

Comparación Rápida. Resumiendo las tres bibliotecas vistas:

Librería	Síncrona	Asíncrona	HTTP/2
<code>requests</code>	Sí	No	No principalmente
<code>httpx</code>	Sí	Sí	Sí
<code>aiohttp</code>	No principalmente	Sí	No principalmente

USO DE LIBRERÍAS HTTPX Y HTTP/2

Cientes con httpx. Proporciona dos clientes principales.

- Cliente síncrono (`httpx.Client()`). *Bloquea* la ejecución mientras espera la respuesta.

```
import httpx
with httpx.Client() as cliente:
    respuesta = cliente.get("https://example.com")
```

- Cliente asíncrono (`httpx.AsyncClient()`). Permite utilizar *async* y *await*.

```
import asyncio
import httpx
async def consultar():
    async with httpx.AsyncClient() as cliente:
        respuesta = await cliente.get("https://example.com")
        print(respuesta.status_code)
asyncio.run(consultar())
```

Uso de HTTP/2. Se debe solicitar en el *context manager* con (`http2=True`), para que el servidor utilice dicho protocolo. En algunos casos requiere el soporte adicional *poetry add "httpx[http2]"*.

```
with httpx.Client(http2=True) as cliente:
    respuesta = cliente.get("https://example.com")
print(respuesta.http_version) # Para consultar version HTTP usada
```

Si un servidor no puede utilizar el protocolo requerido, `httpx` negocia y utiliza otro como *alternativa*, si el servidor lo permite.

Multiplexación.

HTTP/2 *Permite multiplexear varias solicitudes dentro de una conexión. Permite que múltiples flujos compartan una conexión, lo que puede reducir latencia.*

HTTP/1.1 *Varias solicitudes puede necesitar conexiones separadas o reutilizarse secuencialmente.*

Ejemplo de Configuración de un Cliente:

```
import httpx
with httpx.Client(
    base_url="https://api.example.com",
    timeout=10.0,
    headers={
        "Accept": "application/json",
        "User-Agent": "curso-python",
    },
    http2=True,
) as cliente:
    respuesta = cliente.get("/usuarios")
    respuesta.raise_for_status()
```

Reutilizar Conexiones. Preferible cuando realizarás *varias solicitudes*, lo que evita crear un cliente nuevo para cada solicitud.

```
with httpx.Client() as cliente:
    for ruta in ("/usuarios", "/productos", "/ordenes"):
        respuesta = cliente.get(ruta)
        respuesta.raise_for_status()
```

Cliente Asíncrono (varias). Para varias operaciones concurrentes, útil cuando dichas operaciones de entrada/salida pueden esperar simultáneamente.

```
import asyncio
import httpx
async def consultar_varias():
    async with httpx.AsyncClient(http2=True) as cliente:
        respuestas = await asyncio.gather(
            cliente.get("https://example.com/uno"),
            cliente.get("https://example.com/dos"),
        )
    return respuestas
asyncio.run(consultar_varias())
```

Comparación entre Clientes. Cuando usar cada uno:

Síncrono (<code>httpx.Client</code>)	Asíncrono (<code>httpx.AsyncClient</code>)
▪ Flujo es <i>secuencial</i>. ▪ No utiliza <i>asyncio</i>. ▪ Se necesita <i>simplicidad</i>.	▪ Se necesita <i>conurrencia</i>. ▪ Muchas <i>solicitudes independientes</i>. ▪ Se busca <i>evitar bloqueos al esperar</i>.

USO DE TIMEOUTS, REINTENTOS Y ERRORES

Timeouts. *Limita* cuánto tiempo puede esperar una operación de red. Se utiliza en el contexto manager con `httpx.Client(timeout=timeout)`.

Ejemplo	Tipos Principales
<pre>import httpx timeout = httpx.Timeout(connect=5.0, read=10.0, write=10.0, pool=5.0,)</pre>	▪ <i>connect</i>: tiempo para conectarse al servidor. ▪ <i>read</i>: tiempo para recibir datos. ▪ <i>write</i>: tiempo para enviar datos. ▪ <i>pool</i>: tiempo para obtener una conexión disponible.

Excepciones de httpx. Algunas excepciones importantes que pueden capturarse:

```
httpx.ConnectTimeout
httpx.ReadTimeout
httpx.ConnectError
httpx.ReadError
httpx.HTTPStatusError
httpx.RequestError
```

Reintentos. Siempre debe existir un *límite de intentos*. No todos los errores deben reintentarse.

Pueden reintentarse	No deben reintentarse
▪ <i>Timeouts.</i> ▪ <i>Errores de conexión.</i> ▪ <i>429 Too Many Requests</i> ▪ <i>Algunos errores 5xx.</i>	▪ <i>400 Bad Request</i> ▪ <i>401 Unauthorized</i> ▪ <i>403 Forbidden</i> ▪ <i>404 Not Found</i>

Backoff. Espera un tiempo *creciente* entre intentos. Puede añadirse variación aleatoria llamada *jitter*, para evitar que muchos clientes reintenten al mismo tiempo.

Idempotencia. Los reintentos son *más seguros* con métodos idempotentes, como *GET*, *PUT* y *DELETE* (según el diseño de la API). Un *POST* puede crear duplicados si se reenvía sin mecanismos como una clave de idempotencia.

STREAMING DE RESPUESTAS

Streaming. Permite procesar una respuesta por partes, sin cargar todo su contenido en memoria. Es decir, los datos se reciben progresivamente.

```
with cliente.stream("GET", url) as respuesta:
    for bloque in respuesta.iter_bytes():
        ...      # NOTA - .iter_bytes produce bloques de bytes
```

Descargar a Disco. En su argumento usa *"wb"*, que significa: *w* - escritura, *b* - binario.

```
from pathlib import Path
import httpx
destino = Path("descargas/archivo.bin")
destino.parent.mkdir(parents=True, exist_ok=True)

with httpx.Client() as cliente:
    with cliente.stream("GET", url) as respuesta:
        respuesta.raise_for_status()

        with destino.open("wb") as archivo:
            for bloque in respuesta.iter_bytes():
                archivo.write(bloque)
```

Tamaño de Bloque. Se puede indicar el tamaño con *chunk_size*. Un *bloque pequeño* utiliza menos memoria, pero puede requerir más operaciones de lectura y escritura. Uno *demasiado grande* puede consumir más memoria.

```
for bloque in respuesta.iter_bytes(chunk_size=64 * 1024): #Bloque 64bits
    archivo.write(bloque)
```

Verificar antes de escribir. Utiliza *raise_for_status* para *validar* la respuesta antes de crear o escribir el archivo, así no se guardan páginas de error.

```
with cliente.stream("GET", url) as respuesta:
    respuesta.raise_for_status()
    with destino.open("wb") as archivo:
        ...
```

Manejo de Errores. Encapsular dentro de un `try-except` ayuda a manejar excepciones y cerrar los *context managers*, en caso de error durante la transferencia.

```
try:
    with cliente.stream("GET", url) as respuesta:
        respuesta.raise_for_status()

        with destino.open("wb") as archivo:
            for bloque in respuesta.iter_bytes():
                archivo.write(bloque)
except httpx.HTTPError:
    print("Error durante la descarga")
```

Archivos Parciales. Se puede crear archivos incompletos si la descarga falla. Para ello se puede descargar a un *archivo temporal*, y al terminar correctamente *renombrar*.

Ventaja de Memoria. Si el archivo tiene 1 GB:

- **Sin streaming:** Se puede intentar cargar 1 GB en memoria.
- **Con streaming:** Solo se mantiene un bloque en memoria cada vez.

SMOCKER

No es una biblioteca de Python. Es una *herramienta* externa para *simular* APIs HTTP y controlar sus respuestas durante las pruebas.

Smocker permite definir:

- Rutas.
- Métodos HTTP.
- Códigos de respuesta.
- Encabezados.
- Cuerpo de la respuesta.
- Secuencias de respuestas para simular reintentos.

Conceptos:

- **Stub.** Respuesta simulada para una solicitud concreta. Puede definir: Método HTTP, ruta, código de respuesta, encabezados, cuerpo, número de veces que se devuelve una respuesta, y secuencia de respuestas para simular reintentos.
- **Uso de Docker.** Ejecuta Smocker dentro de un contenedor aislado. Se encarga de: Descargar la imagen de Smocker, crear el contenedor, ejecutar el servicio, exponer sus puertos, mantener el entorno reproducible, y evitar una instalación manual de Smocker en Windows.

PREPARACIÓN DEL LABORATORIO

➤ Archivos:

- **cliente_http.py**: Contendrá cliente, timeouts, reintentos, backoff y manejo de excepciones.
- **descarga.py**: Streaming, escritura por bloques, archivo temporal y renombrado al completar.
- **main.py**. Integrará las operaciones y mostrará los resultados.

➤ Flujo del programa:

```
Construir rutas con pathlib
↓
Leer el CSV
↓
Convertir cantidades y precios
↓
Parsear fechas
↓
Convertir fechas a una zona local
↓
Calcular métricas
↓
Exportar métricas a JSON
↓
Registrar eventos con logging
```

➤ Métricas por calcular:

- Número de ventas.
- Unidades vendidas.
- Ingreso total.
- Producto con mayor ingreso.
- Fecha de generación.
- Zona horaria utilizada.

➤ Archivos de salida:

- Resultado en: **salidas/metricas.json**
- Logs escritos en: **logs/ventas.log**

➤ Errores por manejar:

- CSV inexistente.
- CSV vacío.
- Columnas faltantes.
- Cantidad no numérica.
- Precio no numérico.
- Fecha con formato inválido.
- Error al escribir el JSON.

- Relación con etapas anteriores:
 - leer_ventas: creado en src/ingesta.py.
 - calcular_metricas: creado en src/metricas.py.
 - exportar_metricas: creado en src/exportacion.py.
 - Uso de pathlib, logging, CSV y JSON.